

E-commerce Práctica

Documentación técnica

Arquitectura hexagonal con Node.js, Express, Prisma, PostgreSQL y React

Repositorio en GitHub

<https://github.com/Rorre12/ecommerce-practica>

```
git clone https://github.com/Rorre12/ecommerce-practica.git
```

Autor: Eliann Grajales Grene

Curso: Taller 3

Fecha: 23 de septiembre de 2026

Contenido

1. Introducción y objetivos
2. Repositorio y tecnologías
3. Arquitectura hexagonal
4. Estructura del proyecto
5. Modelo de datos
6. Reglas de negocio
7. Especificación de la API REST
8. Frontend
9. Instalación y ejecución
10. Pruebas realizadas
11. Conclusiones

El código fuente completo está disponible en: <https://github.com/Rorre12/ecommerce-practica>

1. Introducción y objetivos

Este proyecto es un comercio electrónico minimalista desarrollado como práctica universitaria. Permite registrar usuarios, iniciar sesión, administrar un catálogo de productos con precio y stock, y crear pedidos que descuentan el inventario de forma segura.

El foco principal es aplicar la **arquitectura hexagonal** (puertos y adaptadores) en el backend, de modo que las reglas de negocio queden aisladas de los detalles técnicos como el framework HTTP, el ORM o la librería de cifrado.

Objetivos

- Diseñar un backend en capas: dominio, aplicación e infraestructura.
- Persistir los datos en PostgreSQL mediante Prisma ORM.
- Proteger las contraseñas con bcryptjs y las rutas con tokens JWT y roles (ADMIN / CUSTOMER).
- Validar el stock al crear pedidos y evitar inventario negativo ante pedidos concurrentes.
- Construir una SPA en React que consuma la API con *fetch* (módulos de Auth, Productos y Pedidos).

2. Repositorio y tecnologías

Repositorio: <https://github.com/Rorre12/ecommerce-practica>

Clonar: `git clone https://github.com/Rorre12/ecommerce-practica.git`

Capa	Tecnología	Uso
Backend	Node.js 18+ · Express 4	Servidor HTTP y API REST
ORM	Prisma 5	Modelado, migraciones y acceso a datos
Base de datos	PostgreSQL 16	Persistencia (docker-compose incluido)
Seguridad	bcryptjs · jsonwebtoken	Hash de contraseñas y tokens de sesión
Otros	cors · dotenv	CORS y variables de entorno
Frontend	React 18 · Vite 5 · CSS	SPA con tres módulos

3. Arquitectura hexagonal

La arquitectura hexagonal organiza el sistema en anillos. El **dominio** está en el centro y no depende de ninguna librería externa; define **puertos** (interfaces) que describen lo que necesita del exterior. La **infraestructura** aporta **adaptadores** que implementan esos puertos (Prisma, bcrypt, JWT) o que exponen la aplicación al exterior (Express).

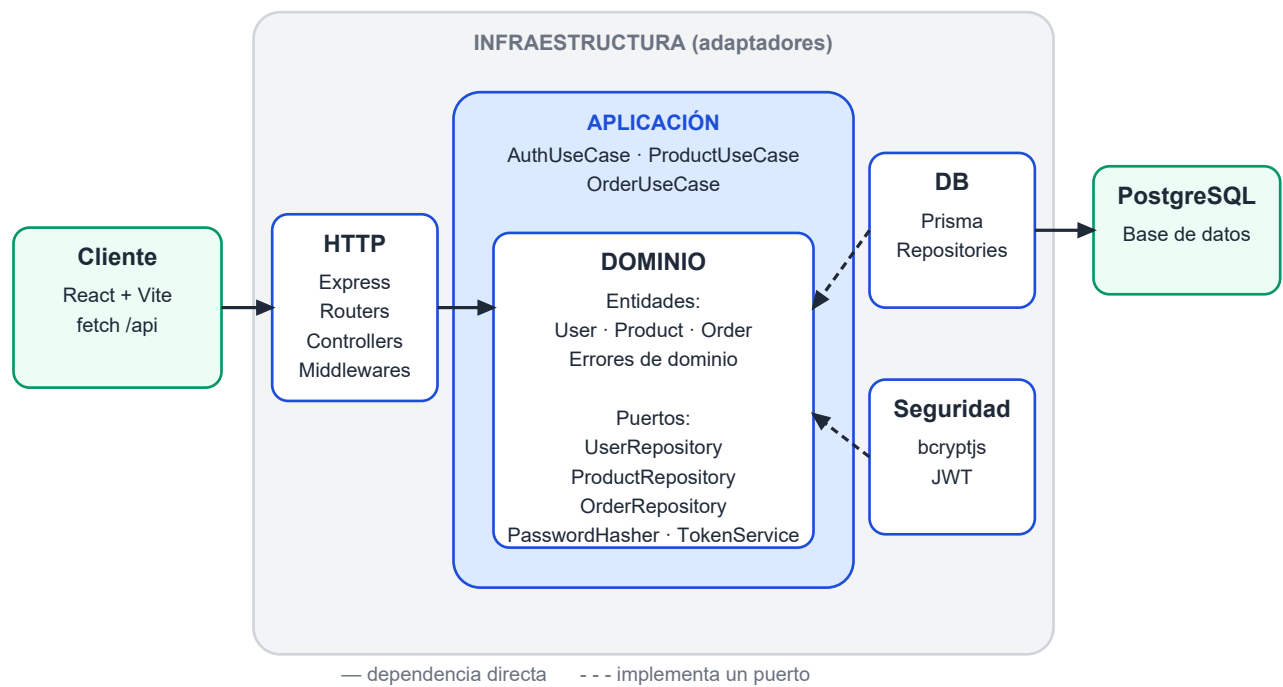


Figura 1. Capas del backend y dirección de las dependencias.

Capas

Capa	Carpeta	Responsabilidad
Dominio	src/domain	Entidades User, Product y Order con sus validaciones; verificación de stock; hash de contraseña a través del puerto PasswordHasher; errores de dominio; definición de puertos.
Aplicación	src/application	Casos de uso AuthUseCase (register, login, me), ProductUseCase (CRUD) y OrderUseCase (create, list, getById). Orquestan dominio y puertos.
Adaptador HTTP	infrastructure/adapters/http	Routers, controllers y middlewares de Express (autenticación, autorización por rol, manejo de errores).
Adaptador DB	infrastructure/adapters/db	Repositorios Prisma que implementan UserRepository, ProductRepository y OrderRepository.
Adaptador seguridad	infrastructure/adapters/security	BcryptPasswordHasher y JwtTokenService.
Composición	src/index.js	Único punto donde se crean los adaptadores concretos y se inyectan en los casos de uso.

Puertos definidos

Puerto	Métodos	Adaptador
UserRepository	findByEmail, findById, save	PrismaUserRepository
ProductRepository	findAll, findById, findByIds, create, update, delete	PrismaProductRepository
OrderRepository	create, findById, findAll, findById	PrismaOrderRepository
PasswordHasher	hash, compare	BcryptPasswordHasher
TokenService	sign, verify	JwtTokenService

Regla de dependencias: el dominio no importa nada de las otras capas; la aplicación solo importa del dominio; la infraestructura implementa los puertos del dominio. Gracias a esto, reemplazar Prisma por otro ORM o bcrypt por otro algoritmo solo requiere un nuevo adaptador y un cambio en `src/index.js`.

4. Estructura del proyecto

```
ecommerce-practica/
├── README.md
├── Documentacion-Ecommerce-Practica.pdf
├── docker-compose.yml          PostgreSQL local
├── docs/                       Documentación en Markdown
├── server/
│   ├── prisma/
│   │   ├── schema.prisma      User, Product, Order, OrderItem
│   │   └── seed.js            Admin y productos de ejemplo
│   └── src/
│       ├── index.js           Raíz de composición
│       ├── domain/
│       │   ├── errors.js
│       │   ├── entities/      User · Product · Order
│       │   └── ports/         Repositorios · PasswordHasher · TokenService
│       ├── application/       AuthUseCase · ProductUseCase · OrderUseCase
│       └── infrastructure/adapters/
│           ├── http/          app · routes · controllers · middlewares
│           ├── db/            prismaClient · Prisma*Repository
│           └── security/       BcryptPasswordHasher · JwtTokenService
└── client/
    ├── vite.config.js         Proxy /api → localhost:4000
    └── src/
        ├── App.jsx · api.js · format.js · styles.css
        └── modules/
            ├── auth/AuthView.jsx
            ├── products/ProductsView.jsx
            └── orders/OrdersView.jsx
```

5. Modelo de datos

El esquema se define en `server/prisma/schema.prisma` para PostgreSQL.

Entidad (tabla)	Campos	Relaciones
User (users)	id, email (único), password (hash bcrypt), role (ADMIN CUSTOMER), createdAt	1 usuario → N pedidos
Product (products)	id, name, price Decimal(10,2), stock, createdAt, updatedAt	1 producto → N líneas de pedido (borrado restringido)
Order (orders)	id, userId, total Decimal(12,2), createdAt	N:1 con User; 1 → N OrderItem
OrderItem (order_items)	id, orderId, productId, quantity, price Decimal(10,2)	N:1 con Order (cascada) y con Product

Precio congelado: OrderItem.price guarda el precio vigente al momento de la compra, por lo que el historial no cambia si después se edita el producto. Los montos usan tipo Decimal para evitar errores de coma flotante.

6. Reglas de negocio

- Las contraseñas se validan (6 a 72 caracteres) y se guardan hasheadas con bcryptjs; nunca se devuelven en la API.
- El registro público crea siempre usuarios CUSTOMER. El administrador inicial se crea con el script de seed.
- Solo ADMIN puede crear, editar o eliminar productos. No se puede eliminar un producto que ya forma parte de pedidos.
- Un pedido debe tener al menos un producto con cantidades enteras mayores a cero; si un producto se repite, se suman las cantidades.
- El dominio verifica que exista stock suficiente para cada producto (`Product.assertStock`).
- El repositorio descuenta el stock dentro de una transacción con `UPDATE ... WHERE stock >= cantidad`. Si otro pedido consumió el stock al mismo tiempo, se lanza `INSUFFICIENT_STOCK` y se revierte todo.
- El total se calcula en centavos y el precio de cada línea queda congelado.
- ADMIN ve todos los pedidos; CUSTOMER solo los propios.

7. Especificación de la API REST

Base URL: `http://localhost:4000/api`. Las rutas protegidas requieren el encabezado `Authorization: Bearer <token>`, obtenido en login o registro.

Método	Ruta	Acceso	Descripción
GET	<code>/api/health</code>	Público	Estado de la API
POST	<code>/api/auth/register</code>	Público	Registrar cliente → 201 { user, token }
POST	<code>/api/auth/login</code>	Público	Iniciar sesión → 200 { user, token }
GET	<code>/api/auth/me</code>	Autenticado	Datos del usuario actual
GET	<code>/api/products</code>	Público	Listar productos
GET	<code>/api/products/:id</code>	Público	Obtener un producto
POST	<code>/api/products</code>	ADMIN	Crear producto { name, price, stock } → 201
PUT	<code>/api/products/:id</code>	ADMIN	Editar producto (campos parciales)
DELETE	<code>/api/products/:id</code>	ADMIN	Eliminar producto → 204
POST	<code>/api/orders</code>	Autenticado	Crear pedido { items: [{ productId, quantity }] } → 201
GET	<code>/api/orders</code>	Autenticado	Historial (ADMIN: todos; CUSTOMER: propios)
GET	<code>/api/orders/:id</code>	Autenticado	Detalle de un pedido (dueño o ADMIN)

Ejemplo: crear un pedido

```
POST /api/orders
Authorization: Bearer eyJhbGciOi...
{ "items": [ { "productId": 1, "quantity": 2 }, { "productId": 2, "quantity": 1 } ] }

201 Created
{ "id": 1, "userId": 2, "userEmail": "cliente@correo.com", "total": 139.3,
  "createdAt": "2026-09-23T15:00:00.000Z",
  "items": [
    { "productId": 1, "productName": "Teclado mecánico", "quantity": 2, "price": 59.9 },
    { "productId": 2, "productName": "Mouse inalámbrico", "quantity": 1, "price": 19.5 } ] }
```

Errores

Todas las respuestas de error tienen el formato { "error": { "code", "message", "details" } }. El adaptador HTTP traduce cada código de dominio a un estado HTTP:

Código	HTTP	Caso
VALIDATION_ERROR	400	Datos inválidos
UNAUTHORIZED	401	Sin token, token inválido o credenciales incorrectas
FORBIDDEN	403	El rol no tiene permiso
NOT_FOUND	404	Recurso inexistente o pedido de otro usuario

Código	HTTP	Caso
CONFLICT	409	Email duplicado o producto con pedidos
INSUFFICIENT_STOCK	409	Cantidad mayor al stock disponible

8. Frontend

SPA en React + Vite ubicada en `client/`. Se comunica con la API mediante *fetch* (archivo `api.js`), que agrega el token JWT y convierte los errores de la API en mensajes visibles.

Módulo	Funcionalidad
Auth	Formulario de inicio de sesión y registro. Guarda la sesión en <code>localStorage</code> y la cierra automáticamente si el token expira (401).
Productos	Tabla con nombre, precio y stock. El ADMIN puede crear, editar y eliminar productos desde un formulario en la misma vista.
Pedidos	Selector de producto + cantidad que arma un carrito, valida el stock y confirma el pedido. Incluye historial con fecha, productos y total (el ADMIN ve también el cliente).

9. Instalación y ejecución

Requisitos: Node.js 18+, PostgreSQL 14+ (o Docker) y Git.

```
# 1. Clonar el repositorio
git clone https://github.com/Rorre12/ecommerce-practica.git
cd ecommerce-practica

# 2. Base de datos (Docker)
docker compose up -d

# 3. Backend
cd server
copy .env.example .env          # ajustar DATABASE_URL y JWT_SECRET
npm install
npx prisma migrate dev --name init
npm run seed                    # admin@tienda.com / Admin123!
npm run dev                     # http://localhost:4000

# 4. Frontend (otra terminal)
cd client
npm install
npm run dev                     # http://localhost:5173
```

Variable	Descripción
DATABASE_URL	Cadena de conexión a PostgreSQL
PORT	Puerto de la API (4000)
CORS_ORIGIN	Origen permitido del frontend (http://localhost:5173)
JWT_SECRET / JWT_EXPIRES_IN	Secreto y duración de los tokens
ADMIN_EMAIL / ADMIN_PASSWORD	Credenciales del administrador creado por el seed

10. Pruebas realizadas

Se verificó que el esquema de Prisma es válido (`prisma validate`) y que el frontend compila sin errores (`vite build`). Además, se levantó la aplicación Express real con los casos de uso reales y repositorios en memoria, y se ejecutaron peticiones HTTP de extremo a extremo:

Escenario	Resultado esperado	Obtenido
Login de administrador	200 con token	200
Login con contraseña incorrecta	401	401
Registro de cliente / email repetido / datos inválidos	201 / 409 / 400	201 / 409 / 400
Cliente intenta crear producto	403	403
ADMIN crea y edita producto	201 / 200	201 / 200

Escenario	Resultado esperado	Obtenido
Pedido con producto repetido (2 + 1 unidades)	201, cantidades sumadas, stock descontado	201
Pedido que supera el stock	409 INSUFFICIENT_STOCK	409
Pedido con producto inexistente	404	404
Historial de pedidos del cliente	200 solo pedidos propios	200
Token inválido	401	401
Eliminar producto (ADMIN)	204	204

Nota: los repositorios Prisma deben probarse contra una instancia real de PostgreSQL siguiendo la sección 9.

11. Conclusiones

- La arquitectura hexagonal separa claramente las reglas de negocio (stock, precios, credenciales) de las tecnologías, lo que facilita probarlas y cambiarlas.
- Los puertos permiten sustituir la base de datos o el mecanismo de cifrado sin modificar el dominio ni los casos de uso.
- La transacción con descuento condicional de stock garantiza la consistencia del inventario incluso con pedidos simultáneos.
- El frontend en React demuestra el consumo completo de la API: autenticación, CRUD de productos y creación e historial de pedidos.

Código fuente: <https://github.com/Rorre12/ecommerce-practica>